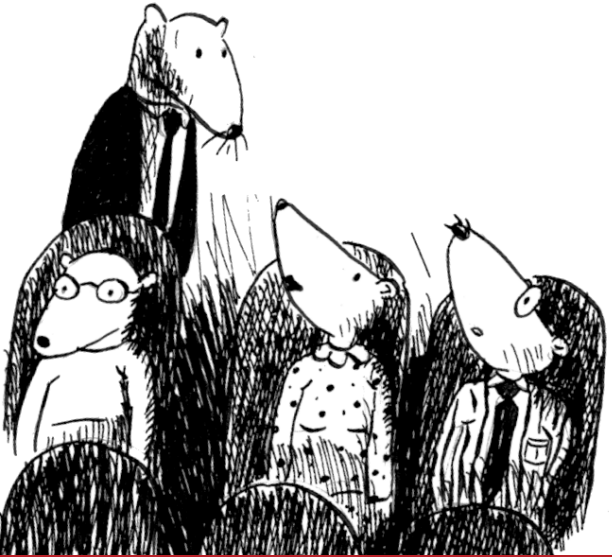


Capítulo 5 — Aplicações para Desenvolvimento de Software

Eduardo Camargo de Siqueira

Tecnologia em Análise e Desenvolvimento de Sistemas

INSTITUTO FEDERAL
Brasília
Campus São Sebastião



drawn by adit.io

Este material foi criado por Eduardo Camargo de Siqueira e está disponível livremente para uso não comercial, para fins didáticos. Se você usar alguma parte deste material, adicione "copyright duca.pro".

Se você encontrar algum erro ou tiver alguma dúvida, envie um e-mail: eduardo.siqueira@ifb.edu.br. Obrigado por contribuir e espero que você aprenda bastante com este material!

Objetivos do capítulo

- relacionar sistemas computacionais e desenvolvimento de software;
- compreender o ambiente de execução de aplicações;
- utilizar terminal, editor de código, Git e servidores locais;
- preparar um ambiente básico de desenvolvimento.

O computador como ambiente de desenvolvimento

Para o desenvolvedor, o computador é um ambiente de:

- criação;
- execução;
- teste;
- depuração;
- publicação de aplicações.

Ideia central

Desenvolver software não é apenas escrever código.

O que compõe um ambiente de desenvolvimento?

- sistema operacional;
- editor de código;
- terminal;
- linguagens;
- bibliotecas;
- gerenciadores de pacotes;
- servidores locais;
- bancos de dados;
- Git;
- contêineres.

Sistema operacional

O sistema operacional é a base do ambiente de desenvolvimento.

- executa editores e IDEs;
- cria processos;
- gerencia arquivos;
- controla permissões;
- disponibiliza terminal;
- permite instalação de pacotes;
- gerencia rede;
- executa VMs e contêineres.

Exemplo: executando uma aplicação

Ao executar:

Terminal

```
python app.py
```

o sistema operacional:

- cria um processo;
- reserva memória;
- acessa arquivos do projeto;
- abre uma porta de rede;
- entrega requisições à aplicação.

Editor de código

Um editor de código ajuda a escrever, organizar e modificar arquivos.

- destaque de sintaxe;
- autocompletar;
- terminal integrado;
- integração com Git;
- formatação automática;
- extensões;
- depuração;
- busca em múltiplos arquivos.

Atenção

O editor ajuda, mas não substitui o entendimento do código.

Terminal

O terminal é essencial para desenvolvimento.

- executar comandos;
- iniciar programas;
- instalar pacotes;
- navegar por diretórios;
- usar Git;
- testar APIs;
- controlar processos;
- iniciar servidores locais.

Terminal: fluxo básico

Criando e executando um projeto

```
mkdir meu-projeto
cd meu-projeto
python3 -m venv venv
source venv/bin/activate
pip install flask
python app.py
```

O terminal evidencia

caminhos, arquivos, processos, permissões, portas e mensagens de erro.

Interpretadores, compiladores e runtimes

Linguagens precisam de ferramentas para executar código.

- **interpretador**: executa código diretamente ou por etapas;
- **compilador**: traduz código antes da execução;
- **runtime**: ambiente necessário para executar programas.

Exemplos de execução

Python

```
python programa.py
```

C

```
gcc programa.c -o programa
./programa
```

Java

```
javac Programa.java
java Programa
```

Gerenciadores de pacotes

Ferramentas para instalar, atualizar e remover dependências.

Ferramenta	Ecossistema	Uso
`pip`	Python	bibliotecas Python
`npm`	Node.js	pacotes JavaScript
`composer`	PHP	dependências PHP
`maven`	Java	projetos Java
`cargo`	Rust	pacotes Rust
`apt`	Debian/Ubuntu	pacotes do SO

Instalando dependências

Python

```
pip install flask
```

Node.js

```
npm install express
```

Cuidado

Dependências desatualizadas, vulneráveis ou incompatíveis podem gerar problemas de segurança e manutenção.

Interpretadores e compiladores

O processador não entende diretamente linguagens como:

- Python;
- Java;
- C;
- JavaScript.

Ideia

O código precisa ser traduzido, interpretado ou executado em um ambiente apropriado.

Linguagens compiladas

Em linguagens compiladas, o código-fonte é traduzido antes da execução.

- há etapa de compilação;
- muitos erros são detectados antes da execução;
- podem gerar executáveis;
- oferecem maior controle sobre recursos;
- geralmente têm bom desempenho.

Exemplos

C, C++, Rust, Go.

Linguagens interpretadas

Em linguagens interpretadas, o código é executado por um interpretador.

- facilitam testes rápidos;
- são flexíveis no desenvolvimento;
- dependem de interpretador instalado;
- são comuns em scripts, web e ciência de dados.

Exemplos

Python, Ruby, JavaScript.

Máquinas virtuais de linguagem

Algumas linguagens compilam para bytecode.

- o código-fonte vira bytecode;
- o bytecode roda em uma máquina virtual de linguagem;
- aumenta a portabilidade entre sistemas.

Exemplo

Java usa a JVM: Java Virtual Machine.

Modelos de execução

Modelo	**Como funciona**	**Exemplos**
Compilado	traduz antes da execução	C, Rust, Go
Interpretado	executa por interpretador	Python, Ruby
Bytecode + VM	compila para bytecode	Java, Kotlin, C#

Ambiente local de desenvolvimento

É o conjunto de ferramentas instalado no computador do desenvolvedor.

- linguagem;
- editor;
- terminal;
- banco de dados;
- servidor local;
- dependências;
- variáveis de ambiente;
- Git;
- contêineres;
- arquivos de configuração.

Instalação de linguagens

Verificar se as ferramentas estão instaladas:

Python

```
python3 --version
```

Node.js

```
node --version
npm --version
```

Java

```
java --version
javac --version
```

Variáveis de ambiente

Variáveis de ambiente guardam configurações lidas por programas.

- modo de execução;
- endereço de banco;
- nome de usuário;
- caminhos;
- chaves e tokens;
- portas de servidores.

Ideia

Separar código e configuração.

Variáveis de ambiente: exemplo

Linux

```
export FLASK_ENV=development
export PORT=5000
```

Python

```
import os

porta = os.getenv("PORT", "5000")
print(porta)
```

PATH

`PATH` informa ao sistema onde procurar executáveis.

- `python`;
- `git`;
- `node`;
- `java`;
- outros comandos do terminal.

Erro comum

"Comando não encontrado" pode indicar instalação ausente ou `PATH` incorreto.

Consultando o PATH

Linux

```
echo $PATH
```

Windows PowerShell

```
$env:Path
```

Ambientes virtuais

Ambientes virtuais isolam dependências de um projeto.

- evitam conflitos entre projetos;
- reduzem instalações globais;
- permitem registrar dependências;
- facilitam reprodução do ambiente.

Exemplo

Um projeto pode usar Flask 2.x e outro Flask 3.x.

Ambiente virtual em Python

Criar

```
python3 -m venv venv
```

Ativar no Linux/macOS

```
source venv/bin/activate
```

Ativar no Windows

```
venv\Scripts\activate
```

Registrando dependências

Salvar dependências

```
pip freeze > requirements.txt
```

Instalar em outro ambiente

```
pip install -r requirements.txt
```

Comparação

Ambiente virtual isola dependências da linguagem. Contêiner isola um ambiente mais amplo.

Controle de versão com Git

Git registra o histórico de alterações de um projeto.

- comparar versões;
- recuperar estados anteriores;
- trabalhar em equipe;
- organizar mudanças;
- integrar com GitHub, GitLab e Bitbucket.

Ideia

Git registra decisões técnicas ao longo do projeto.

Repositórios

Um repositório Git é uma pasta com histórico de versões.

Criar repositório

```
git init
```

Ver estado

```
git status
```

Atenção

O diretório `.git` guarda as informações internas do repositório.

Commits

Um commit registra uma mudança no histórico.

Fluxo básico

```
git add .
git commit -m "cria estrutura inicial"
```

Boa prática

Mensagens de commit devem explicar o motivo da mudança.

Branches

Branches são linhas alternativas de desenvolvimento.

Criar branch

```
git branch nova-funcionalidade
```

Mudar para a branch

```
git checkout nova-funcionalidade
```

Atalho moderno

```
git switch -c nova-funcionalidade
```

Por que usar branches?

Branches permitem trabalhar em:

- novas funcionalidades;
- correções;
- experimentos;
- refatorações;
- versões alternativas.

Exemplo

`main`: versão estável.
`login-google`: nova funcionalidade.

Repositórios remotos

Repositórios remotos ficam hospedados em plataformas como GitHub ou GitLab.

Associar remoto

```
git remote add origin URL_DO_REPOSITORIO
```

Enviar mudanças

```
git push origin main
```

Baixar mudanças

```
git pull
```

Servidores locais

Durante o desenvolvimento, aplicações são testadas no próprio computador.

- executam na máquina local;
- escutam em uma porta;
- respondem a requisições;
- simulam parte do ambiente real.

Endereços comuns

```
http://localhost:8000
http://127.0.0.1:5000
```

Localhost

- `localhost` representa a própria máquina;
- `127.0.0.1` é o endereço de loopback;
- útil para testar aplicações antes da publicação;
- não significa que a aplicação está disponível publicamente.

Servidor HTTP simples

Executar servidor

```
python3 -m http.server 8000
```

Depois, acessar:

Navegador

```
http://localhost:8000
```

Atenção

Útil para testes locais, mas não indicado para produção.

Da máquina local para a produção

O caminho profissional envolve ambientes diferentes.

- desenvolvimento local;
- ambiente de testes;
- homologação;
- produção;
- deploy.

Ideia

Cada ambiente tem finalidade e nível de controle próprios.

Desenvolvimento local

Ocorre no computador do desenvolvedor.

- escrever código;
- executar testes iniciais;
- depurar erros;
- experimentar soluções;
- rodar servidores locais.

Exemplo

```
http://localhost:5000
```

Ambiente de testes

Usado para verificar se o sistema funciona corretamente.

- testes automatizados;
- regras de negócio;
- integração entre componentes;
- detecção de erros antes da publicação.

Exemplos

```
pytest
npm test
```

Homologação

Ambiente intermediário, parecido com produção.

- validação final;
- testes com usuários-chave;
- simulação de uso real;
- aprovação antes da publicação.

Cuidado

Evite usar dados sensíveis reais sem proteção adequada.

Produção

Ambiente real, acessado pelos usuários finais.

- estabilidade;
- segurança;
- desempenho;
- backup;
- monitoramento;
- controle de alterações.

Atenção

Falhas em produção afetam usuários e dados reais.

Deploy

Deploy é o processo de publicar uma aplicação.
Pode envolver:

- copiar arquivos;
- instalar dependências;
- configurar variáveis;
- reiniciar serviços;
- aplicar migrações;
- atualizar contêineres.

Deploy: exemplos

Deploy tradicional

```
git pull
pip install -r requirements.txt
flask db upgrade
sudo systemctl restart minha-aplicacao
```

Deploy com Docker

```
docker compose pull
docker compose up -d --build
```

Relação com DevOps

DevOps aproxima desenvolvimento e operações.

- colaboração;
- automação;
- integração contínua;
- entrega contínua;
- monitoramento;
- infraestrutura como código;
- uso de Git e contêineres.

Automação

Automação transforma tarefas repetitivas em processos executáveis.

- instalar dependências;
- executar testes;
- formatar código;
- construir imagens Docker;
- fazer deploy;
- gerar documentação;
- realizar backup.

Automação: exemplo

Preparar ambiente Python

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Ideia

O que é repetitivo pode ser documentado, automatizado e executado de forma mais confiável.

Integração contínua

CI é a prática de integrar mudanças com frequência e verificar automaticamente o projeto.

- instalar dependências;
- rodar testes;
- verificar estilo;
- analisar vulnerabilidades;
- construir a aplicação.

Benefício

Detectar problemas antes que cheguem à produção.

Entrega contínua

CD mantém o software sempre em condição de ser publicado.

- processo confiável;
- entrega repetível;
- testes automatizados;
- ambientes bem configurados;
- possibilidade de reversão.

Observação

Entrega contínua não significa necessariamente publicar tudo automaticamente.

Monitoramento

Monitoramento acompanha o funcionamento da aplicação e infraestrutura.

- CPU;
- memória;
- disco;
- tempo de resposta;
- erros;
- logs;
- disponibilidade;
- requisições;
- banco de dados;
- rede.

Por que monitorar?

Sem monitoramento, a equipe descobre problemas tarde demais.

Exemplo

Se uma API fica lenta, o monitoramento pode indicar se o problema está em:

- CPU;
- memória;
- banco de dados;
- rede;
- mudança recente no código.

Atividade prática

Escolha um pequeno projeto e descreva seu caminho até produção.

1. Quais ferramentas seriam usadas localmente?
2. Como as dependências seriam instaladas?
3. O projeto usaria Git?
4. Haveria testes?
5. Haveria homologação?
6. Como seria o deploy?
7. O que seria monitorado?
8. O que poderia ser automatizado?

Perguntas para reflexão

- Por que o computador do desenvolvedor é um ambiente técnico complexo?
- Como sistema operacional, terminal, editor e linguagem trabalham juntos?
- Por que o `PATH` ajuda a resolver erros de comando?
- Por que Git é importante mesmo em projetos individuais?
- Por que uma aplicação que funciona em `localhost` pode falhar em produção?

Síntese

- desenvolvimento envolve ambiente, ferramentas e processos;
- linguagens precisam de interpretadores, compiladores ou runtimes;
- dependências devem ser gerenciadas;
- Git organiza histórico e colaboração;
- servidores locais apoiam testes;
- produção exige deploy, segurança, monitoramento e automação.

FIM

- Dúvidas!?
- http://duca.pro.br/ — eduardo.siqueira@ifb.edu.br
- Ilustrações: copyright Manning Publications, drawn by adit.io
- Obrigado pela atenção!